MovieTrack

Application Microservices Distribuée

Spring Boot, gRPC, Kubernetes et Istio :
d'un service local à une application multi-services sécurisée.

Étudiants : Yacine Benfatma
Vincent Tan

Code : github.com/Yacin-e/movietrack

Encadrant : Benoît Charroux

Année universitaire 2025–2026

Table des matières

1 Introduction **2**

2 Architecture **2**
2.1 Vue d'ensemble . . . 2
2.2 Schéma fonctionnel . . . 3
2.3 Communications . . . 3
2.4 Isolation des données . . . 4

3 Stack technique **4**

4 Parcours du projet **4**
4.1 Étape 1 – Premier microservice local . . . 4
4.2 Étape 2 – Gateway Istio . . . 5
4.3 Étape 3 – Second microservice et communication gRPC . . . 6
4.4 Étape 4 – Persistance MySQL . . . 8
4.5 Étape 5 – Sécurisation du cluster . . . 9
4.5.1 RBAC Kubernetes . . . 9
4.5.2 mTLS strict . . . 10
4.5.3 AuthorizationPolicy . . . 11
4.6 Étape complémentaire – Frontend React . . . 11

5 Difficultés rencontrées **12**
5.1 Incompatibilité du starter gRPC avec Spring Boot 3.3.5 . . . 12
5.2 Faux négatif sur `/api/tracking/full` . . . 13
5.3 Tuning des probes Kubernetes . . . 13
5.4 Synchronisation services – MySQL . . . 13

6 Bilan et perspectives **13**

1. Introduction

Ce projet s'inscrit dans le cadre du module de Programmation Distribuée du Master 1 VMI. L'objectif est de concevoir, déployer et sécuriser une application microservices complète, en empilant progressivement les technologies imposées : conteneurisation Docker, orchestration Kubernetes, communications REST et gRPC, persistance relationnelle et service mesh Istio.

Le sujet retenu est **MovieTrack**, une application de suivi de films. L'utilisateur ajoute des films dans un catalogue, enregistre son avancement de visionnage (à voir, en cours, vu) avec note et commentaire, puis consulte sa watchlist enrichie. Cette application est volontairement minimaliste sur le plan fonctionnel : l'enjeu du projet n'est pas la richesse métier, mais la profondeur de la chaîne technologique mise en œuvre.

Le travail s'est arrêté après la sécurisation complète du cluster (RBAC Kubernetes, mTLS strict, `AuthorizationPolicy` Istio). Le déploiement sur une infrastructure cloud n'a pas été réalisé faute de temps ; l'ensemble du système tourne sur Minikube en local. Un frontend React a été ajouté en complément.

2. Architecture

2.1. Vue d'ensemble

L'application est composée de cinq briques déployées dans le namespace Kubernetes `movietrack` :

- `catalog-service` – microservice Spring Boot exposant une API REST de gestion des films et un serveur gRPC interne ;
- `tracking-service` – microservice Spring Boot exposant une API REST de suivi de visionnage et appelant `catalog-service` en gRPC pour enrichir ses réponses ;
- `frontend` – application React servie par nginx ;
- `mysql-catalog` et `mysql-tracking` – deux instances MySQL indépendantes (une par microservice).

L'ensemble est exposé à l'extérieur du cluster via une `Gateway` Istio et un `VirtualService` qui route les requêtes en fonction du préfixe d'URL.

2.2. Schéma fonctionnel

```
                    Client / Navigateur
                           |
                           v
            +--------------------------------+
            |   Istio IngressGateway         |
            |   (namespace istio-system)     |
            +--------------------------------+
                           |
                VirtualService: movietrack-vs
                           |
      +---------------------+--------------------+
      |                     |                    |
 /api/movies          /api/tracking          /  (frontend)
      |                     |                    |
      v                     v                    v
+-----------+         +-------------+      +-----------+
| catalog   |<--------| tracking    |      | frontend  |
| REST 8080 | gRPC    | REST 8080   |      | nginx 80  |
| gRPC 9090 | 9090    | gRPC client |      +-----------+
+-----------+         +-------------+
      |                     |
      v                     v
+-------------+       +---------------+
| mysql-      |       | mysql-        |
| catalog     |       | tracking      |
| StatefulSet |       | StatefulSet   |
| + PVC 1Gi   |       | + PVC 1Gi     |
+-------------+       +---------------+
```

2.3. Communications

Lien	Protocole	Port	Ressource Istio / K8s
Client → Gateway	HTTP	80	`Gateway/movietrack-gateway`
Gateway → catalog	HTTP	8080	route `/api/movies`
Gateway → tracking	HTTP	8080	route `/api/tracking`
Gateway → frontend	HTTP	80	route catch-all `/`
tracking → catalog	gRPC	9090	`CatalogGrpcClient`
Services → MySQL	MySQL/JDBC	3306	`Service/mysql-*`

Table 1 – Communications inter-composants.

Tout le trafic entre pods porteurs d'un sidecar Istio est chiffré en **mTLS strict**. Les pods MySQL fonctionnent sans sidecar (`1/1` au lieu de `2/2`), le protocole MySQL n'étant pas pris en charge nativement par Envoy dans cette configuration.

2.4. Isolation des données

Chaque microservice possède sa propre base de données. Cette séparation respecte le principe *database per service* : aucun service ne lit ou n'écrit dans la base de l'autre. La seule manière pour `tracking-service` d'obtenir des informations sur un film est d'appeler explicitement `catalog-service` via gRPC.

3. Stack technique

Couche	Version	Justification
Java	21	LTS récente, compatible Spring Boot 3.3
Spring Boot	3.3.5	Version stable, alignée sur les dépôts de référence du module
Spring Data JPA	gérée par Boot	ORM avec `ddl-auto: update`, swap H2 → MySQL trivial
gRPC	1.64.0	Implémentation `io.grpc:grpc-netty-shaded` brute
Protocol Buffers	3.25.3	Génération via `protobuf-maven-plugin` 0.6.1
MySQL	8.0	Une instance par service en `StatefulSet`
Docker	–	Images publiées sur Docker Hub (`1yacine/movietrack-*`)
Kubernetes (Minikube)	–	Orchestration locale, RBAC natif
Istio	demo profile	Gateway, VirtualService, mTLS, AuthorizationPolicy
React + Vite	18 / 5	Frontend statique servi par nginx

Table 2 – Technologies utilisées.

4. Parcours du projet

Le projet a été construit de façon incrémentale. Chaque étape ajoute une brique technologique sans casser les précédentes. L'état du dépôt Git est balisé par un tag à chaque palier validé.

4.1. Étape 1 – Premier microservice local

La première étape consiste à mettre en place `catalog-service` en Spring Boot 3.3.5 avec persistance JPA (H2 in-memory) et API REST complète sur `/api/movies`. L'entité `Movie` expose les champs `id`, `title`, `director`, `releaseYear`, `genre`, `runtimeMinutes` et `posterUrl`.

Listing 1 – Endpoints REST exposés

```
GET    /api/movies         liste tous les films
GET    /api/movies/{id}    un film par ID
POST   /api/movies         creation
PUT    /api/movies/{id}    mise a jour
DELETE /api/movies/{id}    suppression
```

Le service est ensuite empaqueté dans une image Docker construite en multi-stage (étape 1 : `eclipse-temurin:21-jdk` avec Maven, étape 2 : `eclipse-temurin:21-jre-alpine` avec uniquement le fat-jar). L'image est publiée sur Docker Hub puis déployée sur Minikube via un `Deployment` et un `Service`.

Fichiers clés.

- `catalog-service/src/main/java/com/movietrack/catalog/` (`Movie.java`, `MovieRepository.java`, `MovieController.java`)
- `catalog-service/Dockerfile`
- `k8s/base/namespace.yaml`, `catalog-deployment.yaml`, `catalog-service.yaml`

Validation. Captures d'écran de `kubectl get pods -n movietrack`, de la requête `curl` sur l'API exposée par le `Service`, et du dépôt Docker Hub public listant l'image.

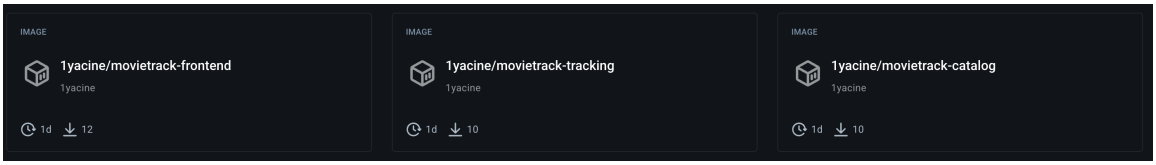

FIGURE 1 – Dépôts publics sur Docker Hub : `movietrack-catalog`, `movietrack-tracking` et `movietrack-frontend`.

```
yacine@Host-004 ~ % kubectl get svc -n movietrack
NAME             TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)                        AGE
catalog          NodePort    10.97.36.163     <none>        8080:31203/TCP,9090:30535/TCP  25h
mysql-catalog    ClusterIP   10.108.138.225   <none>        3306/TCP                       21h
mysql-tracking   ClusterIP   10.99.211.39     <none>        3306/TCP                       21h
tracking         NodePort    10.96.113.230    <none>        8080:30286/TCP                 25h
```

FIGURE 2 – `kubectl get svc -n movietrack` : les Services applicatifs (`NodePort`) et les Services MySQL (`ClusterIP`) à la fin du parcours.

4.2. Étape 2 – Gateway Istio

L'étape suivante installe Istio dans le cluster (profile `demo`), active l'injection automatique des sidecars Envoy sur le namespace `movietrack`, et définit deux ressources Istio :

- `Gateway/movietrack-gateway` – écoute sur le port 80 de l'`istio-ingressgateway`, host `*` ;
- `VirtualService/movietrack-vs` – attache le préfixe `/api/movies` au `Service` catalog.

Une fois les sidecars injectés, les pods passent de `1/1` à `2/2`. Les requêtes externes ne passent plus par le `NodePort` mais par l'IP de l'ingress gateway, ce qui prépare le terrain pour les fonctionnalités de service mesh (mTLS, authorization policies).

Fichiers clés. `k8s/istio/gateway.yaml`, `k8s/istio/virtualservice.yaml`.

```
yacine@Host-004 ~ % kubectl get gateway,virtualservice -n movietrack
NAME                                          AGE
gateway.networking.istio.io/movietrack-gateway   25h

NAME                                            GATEWAYS                HOSTS   AGE
virtualservice.networking.istio.io/movietrack-vs   ["movietrack-gateway"]   ["*"]   25h
```

FIGURE 3 – `Gateway` et `VirtualService` Istio créés dans le namespace `movietrack`.

```
yacine@Host-004 ~ % kubectl get pods -n movietrack
NAME                        READY   STATUS    RESTARTS        AGE
catalog-7f5b5668c9-gcfnj    2/2     Running   1 (6m57s ago)   3h6m
tracking-bcf864465-p5wlt    2/2     Running   0               2m45s
```

FIGURE 4 – Les pods applicatifs passent à `2/2 Running` : le second conteneur est le sidecar Envoy injecté par Istio.

4.3. Étape 3 – Second microservice et communication gRPC

Cette étape ajoute `tracking-service`, qui possède sa propre entité `TrackingEntry` (champs `id`, `movieId`, `status` parmi `TO_WATCH | WATCHING | WATCHED`, `rating`, `notes`, `watchedDate`) et une API REST sur `/api/tracking`.

Le contrat gRPC est défini dans `proto/catalog.proto` et partagé par les deux services :

Listing 2 – Extrait de proto/catalog.proto

```
service CatalogService {
  rpc GetMovie(MovieRequest) returns (MovieResponse);
}
message MovieRequest { int64 id = 1; }
message MovieResponse {
  int64  id              = 1;
  string title           = 2;
  string director        = 3;
  int32  releaseYear     = 4;
  string genre           = 5;
  int32  runtimeMinutes  = 6;
  string posterUrl       = 7;
}
```

`catalog-service` expose un serveur gRPC sur le port 9090 via un `GrpcServerRunner` déclenché en `@PostConstruct`. `tracking-service` expose un client gRPC qui résout l'adresse `catalog:9090` (DNS interne au cluster). L'endpoint `GET /api/tracking/full` retourne la liste des entrées de suivi enrichies, en réalisant un appel gRPC par entrée :

Listing 3 – Réponse JSON enrichie via gRPC

```
[
  {
    "id": 1,
```

```
    "movieId": 1,
    "status": "WATCHED",
    "rating": null,
    "notes": null,
    "watchedDate": null,
    "movie": {
      "id": 1,
      "title": "Inception",
      "director": "Nolan",
      "releaseYear": 2010,
      "genre": "Sci-Fi",
      "runtimeMinutes": 148,
      "posterUrl": ""
    }
  }
]
```


Le `VirtualService` est étendu pour router le préfixe `/api/tracking` vers `tracking-service`.

Choix d'implémentation – gRPC brut. La librairie `net.devh:grpc-spring-boot-starter` a posé des problèmes de compatibilité persistants avec Spring Boot 3.3.5 (boucle de build, classes générées non visibles dans le classpath). Après plusieurs tentatives, nous sommes passés à l'API gRPC brute : `ServerBuilder.forPort(9090)` côté serveur et `ManagedChannelBuilder.forAddress(host, port).usePlaintext()` côté client. Le résultat est plus verbeux mais plus stable, et reste parfaitement compatible avec le sidecar Istio qui intercepte le trafic.

Fichiers clés. `proto/catalog.proto` ; `catalog-service/src/.../grpc/GrpcServerRunner.java` et `CatalogGrpcService.java` ; `tracking-service/src/.../grpc/CatalogGrpcClient.java` ; `k8s/base/tracking-deployment.yaml`, `tracking-service.yaml`.

```
2026-04-24T00:51:00.949Z  INFO 1 --- [          main] c.m.tracking.grpc.CatalogGrpcClient      : gRPC client
connecting to catalog:9090
```

FIGURE 5 – Log de `tracking-service` au démarrage : le client gRPC se connecte à `catalog:9090` (DNS interne du cluster).

```
yacine@Host-004 ~ % kubectl port-forward svc/istio-ingressgateway 29080:80 -n istio-system > /dev/null 2>&1 &
PF_PID=$!
sleep 3
echo "=== /api/movies through gateway ===="
curl -s http://localhost:29080/api/movies
echo ""
echo "=== /api/tracking/full (gRPC enrichment) ===="
curl -s http://localhost:29080/api/tracking/full
kill $PF_PID 2>/dev/null
[1] 11563
=== /api/movies through gateway ===
[{"id":1,"title":"Inception","director":"Nolan","releaseYear":2010,"genre":"Sci-Fi","runtimeMinutes":148,"posterUrl":""}]
=== /api/tracking/full (gRPC enrichment) ===
[{"watchedDate":null,"notes":null,"movie":{"runtimeMinutes":148,"posterUrl":"","releaseYear":2010,"id":1,"title":"Inception","genre":"Sci-Fi",
us":"WATCHED"}]%                                yacine@Host-004 ~ %
[1]  + terminated  kubectl port-forward svc/istio-ingressgateway 29080:80 -n istio-system >  2>&
```

FIGURE 6 – Requêtes `curl` via la gateway Istio : `/api/movies` retourne la liste des films, `/api/tracking/full` retourne une entrée enrichie avec les détails du film obtenus via gRPC.

```
Bash(curl -s -X POST http://127.0.0.1/api/movies \
      -H "Content-Type: application/json" \
      -d
    '{"title":"Dune","director":"Villeneuve","releaseYear":2021,"genre":"SciFi","runtimeMinutes":155,"posterUrl":"x"}'
     && echo ""
    curl -s -X POST http://127.0.0.1/api/tracking \
      -H "Content-Type: application/json" \
      -d '{"movieId":1,"status":"TO_WATCH","notes":"test"}' && echo ""
    curl -s http://127.0.0.1/api/tracking/full)
  {
    "id": 1,
    "title": "Dune",
    "director": "Villeneuve",
    "releaseYear": 2021,
    "genre": "SciFi",
    "runtimeMinutes": 155,
    "posterUrl": "x"
  }
  {
    "id": 1,
    "movieId": 1,
    "status": "TO_WATCH",
    "rating": null,
    "notes": "test",
    "watchedDate": null
  }
```

FIGURE 7 – Réponse complète de `/api/tracking/full` montrant la structure JSON enrichie : l'entrée de tracking embarque le `movie` retourné par l'appel gRPC.

4.4. Étape 4 – Persistance MySQL

Les deux services sont migrés de H2 (in-memory, non persistante) vers deux instances MySQL 8.0 déployées en `StatefulSet`, chacune avec un `PersistentVolumeClaim` de 1 Gi. Cette séparation respecte le pattern *database per service*.

Les identifiants de connexion sont stockés dans des `Secret` Kubernetes (`mysql-catalog-secret`, `mysql-tracking-secret`) et injectés en variables d'environnement dans les pods MySQL et applicatifs.

Profil Spring `k8s`. Pour basculer entre H2 (développement local) et MySQL (cluster), nous utilisons le mécanisme de profils Spring. Le profil par défaut conserve H2 ; le pod en cluster reçoit `SPRING_PROFILES_ACTIVE=k8s` qui charge `application-k8s.properties` avec le datasource MySQL.

Listing 4 – application-k8s.properties (catalog-service)

```
spring.datasource.url=jdbc:mysql://mysql-catalog:3306/catalogdb
spring.datasource.username=${MYSQL_USER}
spring.datasource.password=${MYSQL_PASSWORD}
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.jpa.hibernate.ddl-auto=update
```

Synchronisation du démarrage. Spring Boot échoue immédiatement si MySQL n'est pas prêt au moment de l'ouverture du pool Hikari. Nous avons ajouté un `initContainer` basé sur `mysql:8.0` qui exécute `mysqladmin ping` en boucle jusqu'à obtenir une réponse positive avant de laisser démarrer le conteneur applicatif. Cette approche est plus fiable qu'un simple `nc` sur le port 3306, qui est ouvert avant que le moteur MySQL ne soit réellement disponible.

Validation. La persistance est vérifiée en supprimant le pod applicatif (`kubectl delete pod`) après création de données : à la recréation du pod, les données restent disponibles, ce qui prouve que la persistance se trouve bien dans le `PersistentVolume` et non dans le pod.

Fichiers clés. `k8s/base/mysql-catalog.yaml`, `mysql-tracking.yaml`, `mysql-secrets.yaml` ; `catalog-service/src/main/resources/application-k8s.properties` et son équivalent pour `tracking-service`.

```
yacine@Host-004 ~ % kubectl get pods -n movietrack -o wide
NAME                        READY   STATUS    RESTARTS      AGE   IP            NODE       NOMINATED NODE   READINESS GATES
catalog-5fb49f7d6f-rcndz    2/2     Running   0             29m   10.244.0.57   minikube   <none>           <none>
mysql-catalog-0             1/1     Running   1 (36m ago)   21h   10.244.0.52   minikube   <none>           <none>
mysql-tracking-0            1/1     Running   1 (36m ago)   21h   10.244.0.49   minikube   <none>           <none>
tracking-7bcbb78b7-7ms52    2/2     Running   0             27m   10.244.0.58   minikube   <none>           <none>
```

FIGURE 8 – `kubectl get pods -o wide` : les deux services applicatifs (`2/2`, sidecar Istio) coexistent avec les deux instances MySQL (`1/1`, sans sidecar).

```
yacine@Host-004 ~ % kubectl get statefulset,pvc -n movietrack
NAME                                 READY   AGE
statefulset.apps/mysql-catalog       1/1     21h
statefulset.apps/mysql-tracking      1/1     21h

NAME                                          STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/mysql-data-mysql-catalog-0   Bound    pvc-2eb80df9-e7d2-4f41-a2d9-fd6b85dec4b7   1Gi        RWO            standard       <unset>                 21h
persistentvolumeclaim/mysql-data-mysql-tracking-0  Bound    pvc-5954d943-5621-481c-ac5e-d34ace4ac5c5   1Gi        RWO            standard       <unset>                 21h
```

FIGURE 9 – `StatefulSet` et `PersistentVolumeClaim` : chaque base MySQL dispose de son volume de 1 Gi en statut `Bound`, ce qui garantit la persistance des données indépendamment du cycle de vie des pods.

4.5. Étape 5 – Sécurisation du cluster

C'est l'étape la plus dense en ressources Kubernetes et Istio. Trois couches de sécurité sont mises en place.

4.5.1. RBAC Kubernetes

Chaque microservice tourne désormais sous un `ServiceAccount` dédié (`catalog-sa`, `tracking-sa`) au lieu du `ServiceAccount default`. Chaque `ServiceAccount` est lié à un `Role` restreint qui autorise uniquement les verbes `get` et `list` sur `configmaps` et `secrets` dans le namespace `movietrack` – aucun accès cross-namespace, aucune écriture.

Listing 5 – Extrait de catalog-rbac.yaml

```yaml
rules:
  - apiGroups: [""]
    resources: ["configmaps", "secrets"]
    verbs: ["get", "list"]
```

```
yacine@Host-004 ~ % kubectl get sa,role,rolebinding -n movietrack
NAME                          AGE
serviceaccount/catalog-sa     29m
serviceaccount/default        25h
serviceaccount/tracking-sa    29m

NAME                                          CREATED AT
role.rbac.authorization.k8s.io/catalog-role   2026-04-25T00:45:58Z
role.rbac.authorization.k8s.io/tracking-role  2026-04-25T00:45:58Z

NAME                                                        ROLE                AGE
rolebinding.rbac.authorization.k8s.io/catalog-rolebinding   Role/catalog-role   29m
rolebinding.rbac.authorization.k8s.io/tracking-rolebinding  Role/tracking-role  29m
```

FIGURE 10 – `ServiceAccount`, `Role` et `RoleBinding` dédiés à chaque microservice : `catalog-sa` et `tracking-sa` remplacent le `ServiceAccount default`.

4.5.2. mTLS strict

Une ressource `PeerAuthentication` nommée `default` en scope namespace impose le mode `STRICT`. Toute communication entre pods porteurs d'un sidecar Istio est désormais chiffrée et authentifiée mutuellement. La rotation des certificats est entièrement gérée par le control plane Istio (`istiod`).

Listing 6 – peer-authentication.yaml

```yaml
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: movietrack
spec:
  mtls:
    mode: STRICT
```

La validation se fait via `istioctl x describe pod`, qui affiche explicitement `Workload mTLS mode: STRICT` pour chaque pod du namespace.

```
yacine@Host-004 ~ % istioctl x describe pod $(kubectl get pod -l app=catalog -n movietrack -o jsonpath='{.items[0].metadata.name}') -n movietrack
Pod: catalog-5fb49f7d6f-rcndz
   Pod Revision: default
   Pod Ports: 8080 (catalog), 9090 (catalog)
--------------------
Service: catalog
   Port: http 8080/HTTP targets pod port 8080
   Port: grpc 9090/GRPC targets pod port 9090
8080:
8080 RBAC policies: ns[movietrack]-policy[catalog-access]-rule[0], ns[movietrack]-policy[catalog-access]-rule[1]
9090:
9090 RBAC policies: ns[movietrack]-policy[catalog-access]-rule[0], ns[movietrack]-policy[catalog-access]-rule[1]
--------------------
Effective PeerAuthentication:
   Workload mTLS mode: STRICT
Applied PeerAuthentication:
   default.movietrack
--------------------
Exposed on Ingress Gateway http://192.168.49.2
VirtualService: movietrack-vs
   Match: /api/movies*
   1 additional destination(s) that will not reach this pod
```

FIGURE 11 – `istioctl x describe pod` sur le pod `catalog` : la sortie regroupe à elle seule plusieurs validations – les ports REST/gRPC sont bien exposés, les `AuthorizationPolicy` sont attachées sur les ports 8080 et 9090, le mode mTLS effectif est `STRICT`, et le pod est joignable depuis l'`IngressGateway`.

```
yacine@Host-004 ~ % istioctl proxy-config secret deployment/catalog -n movietrack | head -20
RESOURCE NAME     TYPE           STATUS     VALID CERT     SERIAL NUMBER                        NOT AFTER                NOT BEFORE
default           Cert Chain     ACTIVE     true           ff231343bacfe85d04f2dd4ad8e2b4f3     2026-04-26T00:46:07Z     2026-04-25T00:44:07Z
ROOTCA            CA             ACTIVE     true           2a95a74c17a58b9f7192dff756ad4506     2036-04-20T23:38:41Z     2026-04-23T23:38:41Z
```

FIGURE 12 – `istioctl proxy-config secret` sur le déploiement `catalog` : le sidecar Envoy possède un certificat actif et valide, signé par la `ROOTCA` d'Istio. La rotation de certificats est gérée automatiquement par `istiod`.

4.5.3. AuthorizationPolicy

Les `AuthorizationPolicy` Istio fonctionnent au niveau de l'identité SPIFFE (qui repose elle-même sur le `ServiceAccount` Kubernetes). Deux politiques sont définies :

Destination	Sources autorisées	Autres
`catalog` (REST + gRPC)	`istio-ingressgateway-service-account` (trafic externe) + `tracking-sa` (gRPC interne)	403
`tracking` (REST)	`istio-ingressgateway-service-account` uniquement	403

TABLE 3 – Matrice d'autorisation d'accès.

Concrètement, un pod déployé dans le namespace avec le `ServiceAccount default` ne peut pas appeler `catalog` : sa requête est interceptée par le sidecar Envoy de `catalog` qui répond `403 Forbidden` avant que la requête atteigne l'application Spring Boot.

Fichiers clés. `k8s/rbac/catalog-rbac.yaml`, `tracking-rbac.yaml` ; `k8s/istio/peer-authentication.yaml` ; `k8s/istio/authorization-policies.yaml`.

```
yacine@Host-004 ~ % kubectl get peerauthentication,authorizationpolicy -n movietrack
NAME                                          MODE     AGE
peerauthentication.security.istio.io/default  STRICT   29m

NAME                                                    ACTION   AGE
authorizationpolicy.security.istio.io/catalog-access    ALLOW    29m
authorizationpolicy.security.istio.io/tracking-access   ALLOW    29m
```

FIGURE 13 – Ressources de sécurité Istio dans le namespace : `PeerAuthentication/default` en mode `STRICT` et les deux `AuthorizationPolicy` `catalog-access` / `tracking-access`.

4.6. Étape complémentaire – Frontend React

Une application React (Vite) est ajoutée comme troisième microservice du namespace. Elle est packagée dans une image Docker multi-stage (build Vite avec `node:20-alpine`, puis service statique avec `nginx:alpine`). Le `VirtualService` est complété par une route catch-all `/` qui pointe vers `frontend:80`, placée après les routes `/api/*` pour respecter la priorité de matching.

Trois vues sont implémentées :

— **Movies** – liste et formulaire d'ajout de films ;

— **Track** – formulaire de création d'une entrée de suivi ;

— **Watchlist** – liste enrichie via `/api/tracking/full` (qui déclenche en interne les appels gRPC).

Le frontend ne nécessite pas de `ServiceAccount` privilégié : il sert uniquement des fichiers statiques et le navigateur appelle directement les API via la gateway Istio.

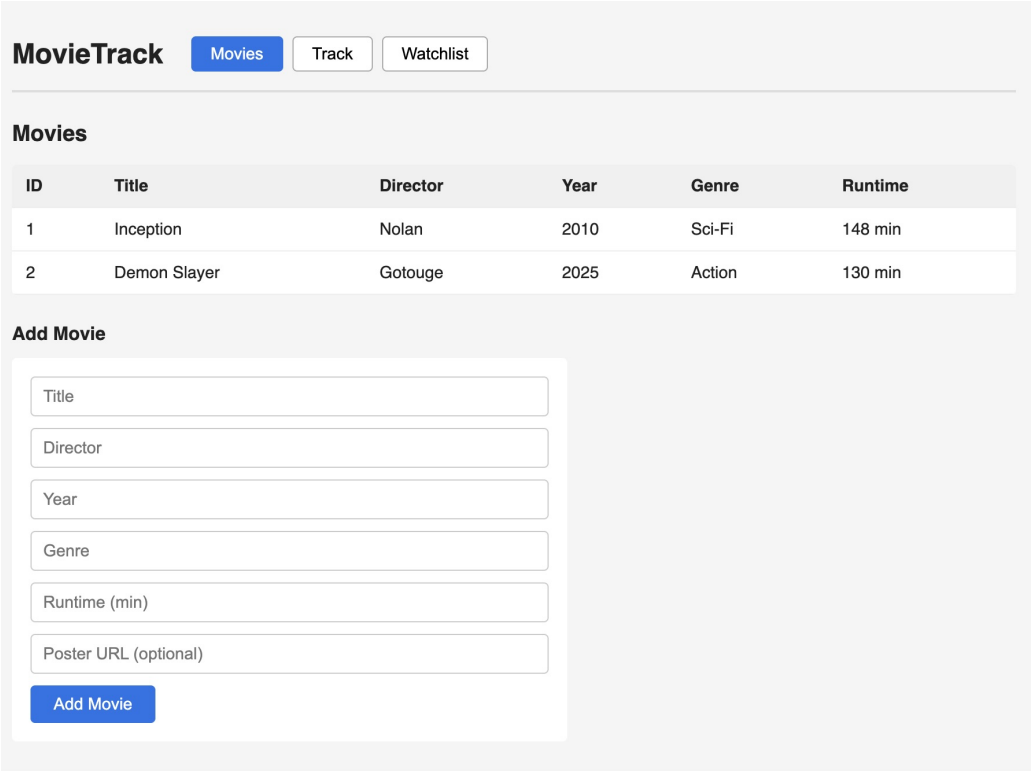

FIGURE 14 – Vue `Movies` du frontend : liste des films issus de `/api/movies` et formulaire d'ajout.

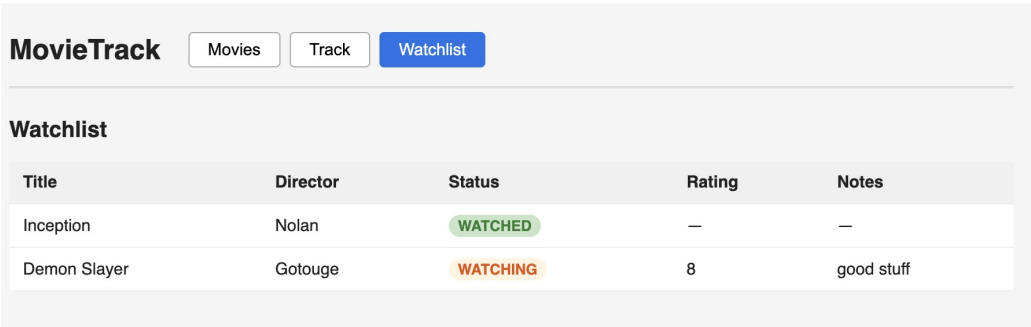

FIGURE 15 – Vue `Watchlist` : entrées de suivi enrichies via `/api/tracking/full` (chaque ligne déclenche en interne un appel gRPC vers `catalog-service`).

5. Difficultés rencontrées

5.1. Incompatibilité du starter gRPC avec Spring Boot 3.3.5

L'intégration initiale de gRPC reposait sur `net.devh:grpc-spring-boot-starter`, recommandée pour le sucre syntaxique qu'elle apporte (`@GrpcService`, `@GrpcClient`).

Plusieurs versions ont été essayées sans succès : la génération des classes Protobuf échouait silencieusement ou les classes générées n'étaient pas visibles depuis le code Spring. Après une demi-heure de boucle, nous sommes passés à l'API gRPC brute (`io.grpc:grpc-netty-shaded` + `ServerBuilder` + `ManagedChannelBuilder`). La configuration est plus verbeuse mais fonctionne immédiatement et reste totalement transparente pour Istio.

5.2. Faux négatif sur `/api/tracking/full`

Lors du premier test de l'enrichissement gRPC, l'endpoint retournait `"movie": null` alors que le serveur gRPC démarrait correctement. Nos premières hypothèses portaient sur mTLS ou la résolution DNS de `catalog:9090`. La vraie cause était plus prosaïque : un redémarrage antérieur du pod `catalog` avait vidé la base H2 in-memory, mais l'entrée `tracking` pointant vers `movieId=1` survivait dans la base `tracking`. L'appel gRPC réussissait techniquement et retournait un film vide, qui était ensuite désérialisé en `null`. Le diagnostic a été établi en activant le log explicite de l'exception côté client gRPC. Cet épisode souligne pourquoi nous avons par la suite migré vers MySQL avec persistance.

5.3. Tuning des probes Kubernetes

Sur Minikube avec une mémoire contrainte, le démarrage de Spring Boot prend de l'ordre de 55 secondes. Avec les valeurs par défaut des `readinessProbe` et `livenessProbe`, le pod était déclaré `Unhealthy` avant la fin du démarrage, ce qui déclenchait un `CrashLoopBackOff`. Nous avons élargi les fenêtres (`initialDelaySeconds: 90` sur la readiness, `initialDelaySeconds: 120` sur la liveness, `failureThreshold: 5`) pour laisser le temps à la JVM de se charger.

5.4. Synchronisation services – MySQL

Au tout premier déploiement avec MySQL, le service Spring Boot tentait d'ouvrir son pool Hikari avant que le moteur MySQL ne soit prêt à répondre, ce qui provoquait un échec fatal. Un simple `nc -z mysql-catalog 3306` dans un `initContainer` ne suffisait pas, car le port 3306 est exposé bien avant que MySQL ne soit pleinement opérationnel. La solution a été d'utiliser un `initContainer` basé sur `mysql:8.0` qui exécute `mysqladmin ping` en boucle jusqu'à recevoir une réponse positive.

6. Bilan et perspectives

Le projet a permis de mettre en œuvre concrètement la chaîne complète d'une application distribuée moderne : un service de développement local, son industrialisation en conteneur, son orchestration en cluster, son ouverture par une gateway, sa duplication en architecture multi-services avec communication inter-services typée (gRPC), sa persistance avec isolation par service, et enfin sa sécurisation par défense en profondeur (RBAC Kubernetes + mTLS + autorisation par identité SPIFFE).

Ce qui n'a pas pu être fait. Le déploiement sur une infrastructure cloud (GKE en particulier) n'a pas été réalisé faute de temps. Les manifestes Kubernetes restent en revanche portables : la migration vers GKE consisterait essentiellement à remplacer le `StorageClass` par défaut par `standard-rwo`, à reconstruire les images en `linux/amd64` et à ajuster le profil d'installation Istio.

Pistes d'extension.

— Observabilité – ajout de Prometheus + Grafana et de Kiali pour visualiser le mesh et les flux mTLS.

— Gestion avancée des secrets – migration vers Sealed Secrets ou External Secrets Operator pour ne plus stocker les identifiants MySQL en clair dans les manifestes.

— Pipeline CI/CD – automatisation des étapes `docker build` et `kubectl apply` via GitHub Actions, avec scan de vulnérabilités sur les images.

— Authentification utilisateur – intégration d'un fournisseur OIDC (Keycloak ou équivalent) au niveau de la gateway, pour passer d'une autorisation pod-à-pod à une autorisation utilisateur-à-pod.

Références

[1] *Spring Boot Reference Documentation*, docs.spring.io/spring-boot.

[2] *Istio Documentation – Security*, istio.io/latest/docs/concepts/security.

[3] *Kubernetes Documentation – Using RBAC Authorization*, kubernetes.io/docs/reference/access-authn-authz/rbac.

[4] *gRPC – A high performance, open source RPC framework*, grpc.io/docs/languages/java.

[5] B. Charroux, *Dépôts pédagogiques de référence : kubernetes-minikube, gRPCSpring, servicemesh-kubernetes, rentalservice*, github.com/charroux.